

Home/Instructions

Please add your documentation for the functions, pipelines, UI, CV/RAG stuff you have built in your tab (basically all the stuff you did from your deliverables). Ideally due by 5/20 so we can look over it and edit anything if necessary. For documentation you have already written, please copy and paste it from the github to your respective tab.

Don't worry about the format, Tejas and I will format it! Thanks!

Zara

1. Transcription Module

The transcription module converts lesson audio into plain text for the AI Lesson Video Feedback Tool. The transcript becomes the first text-based input consumed by downstream processing, including chunking, embedding generation, retrieval, and feedback generation.

Item	Details
Primary file	backend/transcription.py
Main entry point	transcribe_audio(audio_path_or_url: str) -> str
Supported input types	Local audio file paths and remote HTTP/HTTPS audio URLs.
Output	Plain-text transcript string.
Intended use	Called programmatically by upload, video, or feedback pipeline code rather than manually by users in production.

Purpose and role in the pipeline

This module is the first stage of the AI feedback pipeline when the source material is audio or video. It creates the transcript that later steps use as the text representation of the lesson. Keeping this logic in a dedicated backend module makes it reusable from command-line tests, upload handlers, and future pipeline integrations.

Function behavior

- For local file inputs, `transcribe_audio` verifies that the path exists, opens the file in binary mode, and sends it to the OpenAI Audio Transcriptions API.
- For URL inputs, the function downloads the file to a secure temporary location first, because the transcription API expects a file upload rather than a remote URL.
- After a URL-based transcription finishes, the temporary file is removed in cleanup logic so audio files do not accumulate on the server.
- The function returns plain text only. It does not currently return timestamps, speaker labels, or structured segments.

Model choice

Model	Use in this project	Reasoning
gpt-4o-mini-transcribe	Current default	Accurate enough for plain-text transcription and lower cost for the current pipeline needs.
gpt-4o-transcribe-diarize	Considered for later	Supports speaker differentiation and timestamped speaker segments, but the current feedback pipeline does not yet require speaker-level analysis.

Implementation decision

The pipeline currently needs clean transcript text more than diarization or timestamps. Diarization can be added later if feedback requirements expand to speaker-level analysis or time-aligned comments.

URL support helpers

Helper	Responsibility
<code>_is_url(s: str) -> bool</code>	Detects whether the input string is an HTTP or HTTPS URL.
<code>_download_url_to_temp_file(url: str) -> str</code>	Downloads remote audio, validates that the response appears to be audio, preserves the extension when possible, writes to a temporary file, and returns the temporary path.

URL download flow

1. Check whether the input is a URL.
2. Send an HTTP request to download the remote audio file.
3. Validate the response Content-Type to confirm it looks like audio.
4. Create a secure temporary local file, preserving the original extension when possible.
5. Write the downloaded content to the temporary file.
6. Pass the temporary file path to the transcription call.
7. Delete the temporary file after transcription completes or errors.

Development usage and testing

The module can be run directly during local development. In production it is expected to be called by another backend component.

```
python3 -m pip install -r backend/requirements.txt
python3 backend/transcription.py backend/sample_audio.m4a
python3 backend/transcription.py "https://example.com/audio.m4a"
```

Test case	Expected result
Local .m4a file	The module opens the local file and prints/returns a plain-text transcript.
Remote audio URL	The module detects the URL, downloads it, transcribes it, and removes the temporary file afterward.
Missing local file	The function should fail before making a transcription request.
Non-audio URL response	The Content-Type validation should prevent sending the wrong file type to transcription.

2. Admin View API

The Admin View API provides secure endpoints for retrieving instructor-related data. The endpoints are intended for authenticated admin users and support admin workflows such as viewing instructors, reviewing instructor-uploaded files, and accessing feedback records.

Endpoint	Purpose	Expected consumer
GET /api/admin/instructors	Returns instructor records available to the logged-in admin.	Admin instructor list or dashboard sidebar.
GET /api/admin/instructors/:instructorId/files	Returns files uploaded by a specific instructor after instructor validation.	Instructor detail view, file review UI.
GET /api/admin/instructors/:instructorId/feedback	Returns feedback associated with a specific instructor after instructor validation.	Instructor detail view, feedback review UI.

Authentication and authorization

- Every Admin View endpoint requires a logged-in user.
- Admin authorization is handled through a shared helper that verifies the user role before returning protected data.
- Unauthenticated requests return 401 Unauthorized.
- Authenticated users without admin permissions return 403 Forbidden.
- The shared helper keeps authorization behavior consistent across current and future admin endpoints.

Endpoint details

GET /api/admin/instructors

Returns a list of instructors available to the logged-in admin. The response may include instructor ID and email depending on available schema fields.

```
[
  {
    "id": "instructor_id",
    "email": "instructor@example.com"
  }
]
```

GET /api/admin/instructors/:instructorId/files

Returns uploaded files for one instructor. The route validates that the provided instructorId belongs to an existing instructor before retrieving file records.

```
[
  {
    "id": "file_id",
```

```
"name": "example.pdf",
"url": "https://example.com/file.pdf",
"created_at": "2026-05-20T00:00:00Z"
}
]
```

GET /api/admin/instructors/:instructorId/feedback

Returns feedback records associated with a specific instructor. If the instructor does not exist, the endpoint returns 404 Not Found.

```
[
  {
    "id": "feedback_id",
    "feedback": "Feedback text",
    "created_at": "2026-05-20T00:00:00Z"
  }
]
```

Status	Meaning
401 Unauthorized	The user is not logged in.
403 Forbidden	The user is logged in but does not have admin access.
404 Not Found	The requested instructor does not exist.
500 Internal Server Error	Unexpected server or database failure.

3. File Upload and Feedback Generation Pipeline

This implementation connects authenticated PDF uploads to the feedback-generation pipeline. It stores PDFs in Supabase Storage, records metadata in the files table, tracks processing state, and supports generating feedback from uploaded files.

Upload API

Item	Details
Endpoint	POST /api/files/upload
Purpose	Uploads a PDF for the currently authenticated user.
Request format	multipart/form-data
Expected field	file
Success response	201 Created with { "file_id": "uuid" }.
Storage bucket	documents
Storage path format	{user_id}/{file_id}.pdf

Upload authentication

The route creates a server-side Supabase client and checks the current user with `getUser()`. If no authenticated user is found, the route returns 401 and does not attempt to parse, validate, upload, or insert the file.

Validation rules

- A file must be present in the multipart form data.
- Only one file is accepted per request.
- The file cannot be empty.
- The file size must not exceed `MAX_UPLOAD_BYTES`. If `MAX_UPLOAD_BYTES` is unset, the route defaults to 5 MB.
- The MIME type must be `application/pdf`.
- The first bytes of the file must match the PDF magic bytes `%PDF-`.

Why the magic-byte check matters

The browser-provided MIME type is useful, but it is not enough on its own. A file may be labeled as a PDF even if its contents are incomplete, corrupted, or not actually in PDF format. Checking the file's PDF signature gives the backend a second layer of validation before the file is stored or used in the feedback pipeline.

Upload error format

Upload errors use a consistent JSON response shape so the frontend can display readable messages without special-casing every failure.

```
{
  "error": {
    "code": "ERROR_CODE",
    "message": "Readable error message"
  }
}
```

Status	Code	Meaning
400	MISSING_FILE	No file was provided.
400	MULTIPLE_FILES_NOT_ALLOWED	More than one file was provided.
400	EMPTY_FILE	The file is empty.
400	INVALID_PDF	The file does not start with <code>%PDF-</code> .
401	UNAUTHORIZED	The user is not signed in.
413	FILE_TOO_LARGE	The file exceeds <code>MAX_UPLOAD_BYTES</code> .
415	UNSUPPORTED_MEDIA_TYPE	The file is not a PDF.
500	UPLOAD_FAILED	The Supabase Storage upload failed.
500	DATABASE_ERROR	File metadata could not be saved after upload.

Database insert and cleanup behavior

After a successful Storage upload, the route inserts a row into the files table. If the Storage upload succeeds but the database insert fails, the uploaded object is removed from Storage so the system does not leave orphaned files behind.

files column	Purpose
file_id	UUID used to identify the uploaded file throughout the pipeline.
user_id	Owner of the uploaded file.
storage_path	Path to the PDF in Supabase Storage.
original_name	Original filename provided by the user.
content_type	MIME type captured from the upload.
status	Current processing state. New uploads start as uploaded.
status_detail	Optional detail about a processing failure or status note.
created_at / updated_at	Timestamps for creation and latest update.

Status lifecycle

Status	Meaning	Owned by
uploaded	File was uploaded and is ready for processing.	Upload route
processing	Feedback generation has started.	Feedback route
complete	Feedback generation finished successfully.	Feedback route
failed	Feedback generation failed; details may be stored in status_detail.	Feedback route

Expected success path: uploaded -> processing -> complete. Failure path: uploaded -> processing -> failed.

Frontend upload UI expectations

- The user selects a PDF from the UI.
- The frontend creates a FormData object and appends the file under the file field.
- The frontend sends POST /api/files/upload.
- On success, the file list reloads or inserts the new row into visible state.
- On error, the UI parses the JSON error and shows the returned message.

```
const data = await res.json().catch(() => null)
setError(data?.error?.message ?? 'Upload failed.')
```

Feedback generation API

Item	Details
Endpoint	POST /api/feedback/generate
Purpose	Generates feedback from a lesson plan PDF.
Supported input flows	Existing instructorId + lessonPlanId flow and uploaded-file flow using file_id.
Uploaded-file request	{ "file_id": "uuid" }
Success response	{ "success": true, "feedbackId": "uuid", "storagePath": "path/to/generated-feedback.pdf" }

Uploaded-file generation flow

1. Validate that file_id is a valid UUID.
2. Look up the file in the files table.
3. Verify that the requester has access to the file.
4. Confirm the file is currently in uploaded status.
5. Update the file status to processing.
6. Download the PDF from the documents bucket.
7. Extract text from the PDF buffer.
8. Generate feedback using the RAG/OpenAI feedback pipeline.
9. Render the feedback into a PDF.
10. Store the generated feedback PDF and create the feedback row.
11. Update the file status to complete. If an error occurs, update status to failed and store the error detail.

Core feedback functions

Function	Role
extractTextFromPdf	Reads a PDF buffer and extracts lesson-plan text for the pipeline.
getFeedbackFromRag	Generates feedback from extracted lesson text and configured retrieval/OpenAI context.
renderFeedbackPdf	Renders generated feedback into a PDF buffer.
storeFeedbackPdf	Stores the generated PDF and creates the feedback table row, returning feedbackId and storagePath.

Generated feedback retrieval

Generated feedback PDFs are served through GET /feedback/[feedback_id]. The route validates the feedback ID, looks up the feedback row, reads the storage_path, downloads the PDF from feedback storage, and returns the PDF inline with content type application/pdf.

End-to-end PDF flow

1. User uploads a PDF.
2. POST /api/files/upload returns file_id.
3. A files row is created with status uploaded.
4. POST /api/feedback/generate is called with file_id.
5. The files row changes to processing.
6. Feedback is generated and rendered as a PDF.
7. The feedback PDF is stored and the feedback row is created.
8. The files row changes to complete.
9. The feedback can be viewed through GET /feedback/[feedback_id].

Current UI note

The backend supports the uploaded-file `file_id` generation flow. Some UI paths may still use the older `instructorId + lessonPlanId` flow. Uploaded files remain in uploaded status until `POST /api/feedback/generate` is called with their `file_id`.

4. Video Upload UI

Video upload support was added to the instructor dashboard alongside the existing PDF upload flow. The work allows instructors to upload lesson videos, trigger video feedback generation, see long-running status updates, and receive user-friendly errors for invalid or over-limit videos.

Primary file	Responsibility
<code>apps/web/components/InstructorDashboardClient.tsx</code>	Client implementation for PDF and video upload UI, upload handlers, feedback generation, polling, status rendering, and error handling.
<code>apps/web/app/dashboard/instructor/page.tsx</code>	Server-rendered dashboard page for authentication, role gating, initial dashboard data loading, and mapping backend rows into <code>InstructorUploadRow</code> objects.

Frontend architecture

The implementation was built in the newer dashboard UI, `InstructorDashboardClient`, rather than the older chat-style `InstructorWorkspace` flow. It uses row-based state to represent uploaded PDFs, uploaded videos, feedback status, and error information.

PDF upload flow

The existing PDF upload path remains unchanged. The user selects a PDF, the frontend validates the MIME type, creates `FormData`, submits the file, inserts or reloads the dashboard row, and allows the user to trigger feedback generation.

Video upload flow

- The dashboard includes a hidden video input, an Upload Video button, upload state indicators, and upload error messaging.
- The hidden input accepts `video/*` and calls `handleVideoUpload` when a file is selected.
- The handler creates `FormData`, appends the file under `file`, and sends `POST /api/videos/upload`.
- The frontend checks that the selected file is a video but intentionally does not enforce duration locally.
- Video duration enforcement is delegated to the backend upload route so the server remains the source of truth for admin-configured limits.

```

<input
  ref={videoInputRef}
  className="dashboard-file-input"
  type="file"
  accept="video/*"
  onChange={handleVideoUpload}
/>

```

```

const formData = new FormData()
formData.append('file', file)

```

```

fetch('/api/videos/upload', {
  method: 'POST',
  body: formData,
})

```

Upload error handling

Scenario	Frontend behavior
Non-video selected	Rejects the file and displays: Please choose a video file.
Backend duration limit error	Maps a raw duration error into an instructor-friendly message, such as: Video exceeds your admin's limit of 15 minutes.
Unexpected upload failure	Displays a fallback upload error without crashing the dashboard.

Dashboard row state model

```

export type InstructorUploadRow = {
  fileId: string
  fileName: string
  sourceStoragePath: string
  uploadedAt: string | null
  sourceType: 'pdf' | 'video'
  feedbackStatus:
    | 'ready'
    | 'not_started'
    | 'uploaded'
    | 'transcribing'
    | 'generating'
    | 'failed'
  feedbackId: number | null
  errorMessage?: string | null
}

```

Field group	Purpose
-------------	---------

fileId, fileName, sourceStoragePath, uploadedAt	Identify and display each upload row.
sourceType	Distinguishes PDF rows from video rows.
feedbackStatus	Supports PDF status, long-running video status, failed-state rendering, and ready-state feedback access.
feedbackId	Enables View feedback once the generated feedback exists.
errorMessage	Stores row-specific failure text for display.

Feedback generation requests

Source type	Request shape
PDF	{ "instructorId": "...", "lessonPlanId": "...", "originalFilename": "..." }
Video	{ "instructorId": "...", "lessonPlanId": "...", "source_type": "video", "videoFileId": "..." }

The video request shape lets the backend distinguish video jobs from PDF jobs and route video sources through transcription before feedback generation.

Status polling

- The frontend polls GET /api/feedback/user/:userId while active video jobs exist.
- Polling runs approximately every 2.5 seconds while a video row is uploaded, transcribing, or generating.
- Polling stops when the row reaches ready or failed.
- Generate buttons are disabled while the current row is generating or a video pipeline job is active.
- The View feedback button is enabled only when feedbackStatus is ready.

Backend status	UI label
uploaded	Uploaded...
transcribing	Transcribing audio...
generating	Generating feedback...
complete	Feedback ready
failed	Processing failed

Backend dependencies and blockers

- POST /api/videos/upload must accept video uploads, validate MIME type, validate duration limits, persist uploaded video records, and return file_id plus duration_seconds.
- POST /api/feedback/generate must support source_type: video.
- GET /api/feedback/user/:userId must return pipeline status updates for polling.
- ffmpeg runtime availability is required for reliable backend duration checks.
- profiles.max_video_duration_seconds schema support is required for admin-configured duration limits.

5. Persistent Chat Session API

This feature adds backend support for persistent chat sessions in the instructor feedback workspace. It allows a logged-in instructor to start a new chat, view previous sessions, and resume a specific session later.

Table	Important columns	Purpose
chat_sessions	id, user_id, title, created_at, updated_at	Stores one row per chat conversation and supports chat history.
chat_messages	id, session_id, role, content, feedback_id, created_at	Stores individual messages belonging to a chat session.

Row Level Security

- RLS is enabled for both chat_sessions and chat_messages.
- Users can only read, insert, or update their own chat sessions.
- Users can only read or insert messages if the parent session belongs to them.
- Message ownership is enforced through an EXISTS query against chat_sessions.
- Database-level ownership checks supplement route-handler checks.

Authentication pattern

The routes reuse getSessionUser from apps/web/lib/feedback/feedback-route-auth.ts. Each route creates a Supabase server client, calls getSessionUser(supabase), and returns the existing 401 Unauthorized response if the user is not logged in.

```
const supabase = await createClient()
const session = await getSessionUser(supabase)

if (!session.ok) {
  return session.response
}
```

API routes

Endpoint	Behavior	Status codes
POST /api/chat/start	Creates a new empty chat session for the authenticated user. Sets user_id to the logged-in user and title to New chat.	201 created, 401 unauthenticated, 500 creation failed.
GET /api/chat/sessions	Returns sessions owned by the logged-in user, ordered by updated_at descending, with latest-message previews truncated to about 120 characters.	200 loaded, 401 unauthenticated, 500 load failure.

GET /api/chat/sessions/[sessionId]	Validates the UUID, loads a session only if it belongs to the logged-in user, and returns messages ordered by created_at ascending.	200 loaded, 400 invalid UUID, 401 unauthenticated, 404 missing/not owned, 500 load failure.
------------------------------------	---	---

Response shapes

POST /api/chat/start

```
{
  "sessionId": "<uuid>"
}
```

GET /api/chat/sessions

```
{
  "sessions": [
    {
      "id": "<uuid>",
      "title": "New chat",
      "createdAt": "2026-05-20T07:42:05.93231+00:00",
      "updatedAt": "2026-05-20T07:42:05.93231+00:00",
      "lastMessagePreview": null
    }
  ]
}
```

GET /api/chat/sessions/[sessionId]

```
{
  "session": {
    "id": "<uuid>",
    "title": "New chat",
    "createdAt": "<timestamp>",
    "updatedAt": "<timestamp>"
  },
  "messages": [
    {
      "id": "<uuid>",
      "role": "user",
      "content": "Example message",
      "createdAt": "<timestamp>"
    }
  ]
}
```

Authorization details

The session detail route scopes directly by both session ID and user ID. Returning 404 for sessions that are missing or owned by another user avoids revealing whether another user's session ID exists.

```
.eq('id', sessionId)
.eq('user_id', session.user.id)
```

Scope limitation

This deliverable does not add a message insertion endpoint. POST /api/chat/start creates only the parent session. Future chat-send or streaming logic can insert rows into chat_messages while still using these routes for listing and resuming sessions.

Manual tests

Test	Expected result
Create a session	POST /api/chat/start returns 201 and a sessionId.
List sessions	GET /api/chat/sessions returns the created session with lastMessagePreview null when it has no messages.
Invalid session ID	GET /api/chat/sessions/not-a-uuid returns 400 with Invalid sessionId.
Load a new session	GET /api/chat/sessions/<uuid> returns 200 with session metadata and messages: [].

6. CV Research Video Analysis API

CvResearch/main.py was refactored from a run-once script into an importable Python module. The public API analyzes a short Zoom or lesson-video segment, samples frames, runs face and pose detection, and returns structured JSON-compatible results.

Item	Details
Primary file	CvResearch/main.py
Public function	analyze_zoom_segment(...)
Primary output	Dictionary with meta, samples, and aggregates keys.
Primary improvement	Analysis no longer runs automatically on import. The module can be called from CLI, backend route, notebook, or another Python file.

Public function

```
def analyze_zoom_segment(
    video_path: Union[str, Path],
    *,
    max_duration_s: float = 300.0,
```

```
sample_hz: float = 1.0,  
start_offset_s: float = 0.0,  
) -> dict:
```

Parameter	Default	Description
video_path	required	Path to the video file to analyze.
max_duration_s	300.0	Maximum number of seconds to analyze.
sample_hz	1.0	Number of frames sampled per second. 1.0 samples one frame per second; 2.0 samples twice per second.
start_offset_s	0.0	Offset in seconds where analysis should begin.

Return schema

Top-level key	Contents
meta	Video path, backend, FPS, effective duration, sample rate, sample count, wall-clock runtime, and setup error.
samples	One object per sampled frame, including sec, frame_ms, has_face, has_pose, and blendshapes.
aggregates	Reserved for later aggregate statistics across samples; currently returned as an empty dictionary in this documented version.

```
{  
  "meta": {  
    "video_path": "CvResearch/example_video3.mov",  
    "backend": "tasks",  
    "fps": 25.32,  
    "effective_duration_s": 40.2,  
    "sample_hz": 1.0,  
    "sample_count": 40,  
    "total_wall_s": 8.28,  
    "setup_error": ""  
  },  
  "samples": [  
    {  
      "sec": 0.0,  
      "frame_ms": 26.03,  
      "has_face": true,  
      "has_pose": true,  
      "blendshapes": {}  
    }  
  ],  
  "aggregates": {}  
}
```

Internal helper functions

Helper	Responsibility
<code>_load_video(path)</code>	Opens a video file with OpenCV and returns the <code>cv2.VideoCapture</code> object plus FPS. Raises <code>ValueError</code> if the file cannot be opened.
<code>_sample_frames(cap, start_s, end_s, sample_hz)</code>	Generator that yields sampled frames as (sec, frame_bgr). Sampling interval is $1.0 / \text{sample_hz}$.
<code>_process_frame(frame_bgr, backend, face, pose)</code>	Runs face and pose detection on one sampled frame and returns <code>has_face</code> , <code>has_pose</code> , <code>blendshapes</code> , and <code>frame_ms</code> .
<code>ensure_model(model_path, model_url)</code>	Ensures required model files exist locally and downloads missing models when possible.
<code>create_detectors()</code>	Creates MediaPipe Tasks detectors when available or falls back to OpenCV detectors. Returns <code>backend</code> , <code>face</code> , <code>pose</code> , and <code>setup_error</code> .

Side effects removed by the refactor

- No hardcoded video filename is required for import usage.
- No automatic processing starts when importing the module.
- The analysis path does not intentionally write output PNG files.
- The previous `cv2.imwrite` PNG dump loop was removed.
- `annotated_images` accumulation was removed from the API path.
- The public analysis logic returns data instead of relying on print-driven output.

Resource cleanup

`analyze_zoom_segment` uses `try/finally` cleanup. `cap.release()` is called when a video was opened. MediaPipe task detectors are closed when the tasks backend is used. This prevents video handles and detector resources from staying open after analysis completes or errors.

FaceLandmarker / blendshapes note

The current implementation uses MediaPipe FaceDetector, which detects whether a face is present but does not return facial blendshape scores. Because of this, each sample currently includes `blendshapes: {}`. The field remains in the schema so FaceLandmarker integration can populate it later without changing the response shape.

Usage and testing

```
python CvResearch/main.py CvResearch/example_video3.mov
```

```
from CvResearch.main import analyze_zoom_segment
```

```
result = analyze_zoom_segment(  
    "CvResearch/example_video3.mov",  
    max_duration_s=30,
```

```

sample_hz=1.0,
start_offset_s=0.0,
)

```

Test	Expected result
CLI test	Running python CvResearch/main.py <video> prints the JSON result to stdout.
Import test	Importing analyze_zoom_segment does not start analysis, open video files, create detectors, print logs, or write files.
Short analysis test	Calling analyze_zoom_segment with max_duration_s=5 returns meta, samples, and aggregates keys and a sample count matching the sampling config.

7. Cross-Cutting Implementation Notes

Several implementation patterns appear across the deliverables and should be preserved as the project continues.

Pattern	Where it appears	Why it matters
Server-side authentication before work begins	Upload routes, admin routes, chat routes, feedback routes	Prevents unauthenticated requests from reaching storage, database, or model calls.
Ownership checks	Files, chat sessions, admin instructor data	Ensures users can only access their own resources unless explicitly authorized as admins.
Status fields for long-running jobs	files table, video feedback rows	Lets the UI show progress and prevents duplicate generation.
Consistent error shapes	Upload and generation flows	Allows frontend code to surface readable errors without brittle parsing.
Temporary resource cleanup	Transcription URL downloads, Storage upload rollback, CV video handles	Avoids leaked temp files, orphaned storage objects, and open processing resources.
Backward-compatible additions	Feedback generation route, dashboard upload UI	Allows new flows to coexist with existing lesson-plan and PDF workflows.

8. Remaining Dependencies and Follow-Ups

The following items are documented as follow-ups or dependencies rather than completed scope.

- Wire any remaining UI actions that should call POST /api/feedback/generate with file_id for uploaded PDFs.

- Complete and verify backend video-upload dependencies, including route stability, ffmpeg availability, video record persistence, and profiles.max_video_duration_seconds support.
- Ensure GET /api/feedback/user/:userId returns the status fields expected by the dashboard polling loop.
- Add chat-send or streaming logic that inserts rows into chat_messages so the persistent-session APIs can resume full conversations.
- Integrate FaceLandmarker if facial blendshape scores are needed for richer CV analysis.
- Consider diarization or timestamps in transcription only if future feedback requirements need speaker-level or time-aligned feedback.

Colin

Zein

Praneel

Jesus

File Upload Feature

By Jesus Barrios

What Was Built

A PDF upload endpoint for AllStarCode. When a logged-in user sends a PDF to the route, it gets stored in Supabase Storage and a record is saved in the database. That's it, no frontend yet. Just the backend route as requested.

How It Works

The route lives at `POST /api/files/upload`. Here's what happens when it gets hit:

1. Check if the user is logged in. If not, return `401 Unauthorized`.
2. Read the uploaded file from the request form data.
3. Validate it. It must be a PDF, must not be empty, and the first four bytes must match the PDF magic bytes (`%PDF`).
4. Generate a unique file ID and upload the file to Supabase Storage under `documents/{user_id}/{file_id}.pdf`.
5. Insert a row into the `files` table with the file ID, user ID, storage path, original name, and content type.
6. If anything goes wrong after the upload (like the database insert fails), the file gets deleted from storage so nothing is left orphaned.
7. On success, return `201 Created` with `{ "file_id": "..." }`.

Database

A `files` table was created with the following columns: `file_id`, `user_id`, `storage_path`, `original_name`, `content_type`, and `created_at`. Row Level Security is enabled so users can only insert and read their own rows. The same idea applies to Supabase Storage — users can only upload to and read from their own folder.

How to Test

Since the route requires an active session, the easiest way to test is from the browser while already logged into the app. Open the browser console at `localhost:3000` and paste the following:

None

```
const input = document.createElement('input')
```

```
input.type = 'file'
input.accept = 'application/pdf'
input.onChange = async () => {
  const form = new FormData()
  form.append('file', input.files[0])
  const res = await fetch('/api/files/upload', { method: 'POST',
body: form })
  console.log(res.status, await res.json())
}
input.click()
```

This opens a file picker. Select any PDF and the console will print the response. A successful upload returns 201 and a `file_id`. If the user is not logged in it returns 401, and if the file is not a valid PDF it returns 400.

Verification

After a successful upload, two places should reflect it: the `documents` bucket in Supabase Storage should contain a folder named after the user ID with the PDF inside, and the `files` table in the Table Editor should have a new row matching the upload.

Admin Dashboard + File Upload

By Jesus Barrios

What Was Built

Two features were delivered across two branches: a PDF upload endpoint and an admin dashboard UI built on top of it.

File Upload (**feature/file-upload**)

A `POST /api/files/upload` endpoint that lets any logged-in user upload a PDF. The file is validated (non-empty, correct content type, PDF magic bytes), stored in a Supabase Storage bucket called `documents` under a path of `{user_id}/{file_id}.pdf`, and recorded in a `files` table. If the database insert fails after a successful upload, the file is cleaned up from storage automatically. Returns `201` with a `file_id` on success, or `400/401/500` as appropriate.

A `files` table was created with columns `file_id`, `user_id`, `storage_path`, `original_name`, `content_type`, and `created_at`. Row Level Security is on — users can only insert and read their own rows.

Admin Dashboard (**admin-dashboard**)

The admin dashboard is a full redesign of `/admin` using the same sidebar shell as the instructor page. It has two parts.

The first is the instructor list. The sidebar shows all instructors assigned to the logged-in admin (linked via an `assigned_admin_id` column added to the `profiles` table). Each item shows a small avatar circle with the instructor's first initial. Clicking the AllStarCode logo navigates back to the admin home from any instructor detail view.

The second is the instructor detail page at `/admin/instructors/{instructorId}`. It shows an instructor card at the top (avatar, email, truncated ID), then two sections — files and feedback — each with a count. File names are underlined on hover and open the PDF in a new tab via a short-lived Supabase signed URL. Feedback entries show a **view** button that links to `/feedback/{id}`, which is the route built by SWE 4.

The admin also has a self-service section (My Files) where they can upload their own PDFs and see their own file list, using the same upload flow as instructors. File names

here are plain text with no underline, since the file is opened via a click handler rather than a link.

Three API routes back the dashboard: `GET /api/admin/instructors`, `GET /api/admin/instructors/[instructorId]/files`, and `GET /api/admin/instructors/[instructorId]/feedback`. All require the user to be an authenticated admin and verify that the requested instructor is actually assigned to them.

How to Test

Migrations `0004` and `0005` have already been applied. To test the instructor list, go to Table Editor, open the `profiles` table, find an instructor row, and set `assigned_admin_id` to your admin user's ID. Log in as admin and the instructor will appear in the sidebar.

To test file upload from the browser console while logged in:

```
None
const input = document.createElement('input')

input.type = 'file'

input.accept = 'application/pdf'

input.onchange = async () => {

  const form = new FormData()

  form.append('file', input.files[0])

  const res = await fetch('/api/files/upload', { method: 'POST',
body: form })

  console.log(res.status, await res.json())

}

input.click()
```

A successful upload returns `201` with a `file_id`. Unauthenticated requests return `401`, invalid files return `400`.

What Is Not Yet Implemented

Instructor Assignment UI

Assigning an instructor to an admin is currently done manually in the Supabase Table Editor. There is no UI or API route for an admin to claim or be assigned instructors. This needs to be built, likely as a settings or management page under [/admin/settings](#) or a separate admin-only route.

Feedback Generation

The feedback table and generation logic are owned by another member (SWE 3/4). The feedback sections in the dashboard are created and ready. They query [/feedback/{id}](#) and the [GET .../feedback](#) API route, but they will show empty until these members deliver the table and populate it. No changes needed on this side once feedback rows exist.

PDF Viewer Component

The **view** buttons on feedback entries open [/feedback/{id}](#) in a new tab, which streams the PDF as a raw response. SWE 4 is building a proper viewer component. Once delivered, the [href](#) on those buttons can be swapped out or the viewer can be embedded inline. The data is already working.

Instructor File Downloads on Detail Page

File names on the instructor detail page open via a server-generated signed URL (1 hour expiry). If the storage RLS policy for admin reads is not active, the signed URL comes back null and the file name renders as non-clickable text. Confirm that the [admins read instructor storage files](#) policy is present under Storage → Policies in the Supabase dashboard.

Instructor Self-Service (Upload + Feedback)

The instructor page ([/instructor](#)) currently shows a static UI with hardcoded messages and non-functional upload buttons. The upload button needs to be wired to [POST /api/files/upload](#) and the thread needs to reflect real uploaded files and generated feedback. This is a follow-up task.

Instructor Chat Interface

By Jesus Barrios

What Was Built

A full chat interface for AllStarCode instructors, accessible at </dashboard/instructor/chat>. The page has a sidebar listing all past chat sessions and a main window showing the full message history for the selected session. Instructors can start new conversations, type messages, and receive responses from the AI assistant. A typing indicator appears while the response is being generated. A link to the chat page was added to the existing instructor dashboard sidebar.

How It Works

The chat page is a server component that checks authentication the same way the existing instructor dashboard does. It renders a client component called [InstructorChatWorkspace](#) which manages all UI state. The full flow is:

1. On mount, the component fetches [GET /api/chat/sessions](#) to populate the session sidebar, sorted by most recent activity.
2. Clicking a session fetches [GET /api/chat/sessions/{sessionId}](#) to load its full message history. A 404 response clears the selection and refreshes the list.
3. Clicking **+ New Chat** generates a UUID on the client and sets it as the active draft session. No database row is created yet.
4. When the instructor sends a message, the user bubble is added to the thread immediately, a three-dot typing indicator appears, and [POST /api/chat/message](#) is called with the session ID and message text.
5. The API upserts the session row on first message (so no separate session creation endpoint is needed), persists the user turn, calls the AI, persists the assistant turn, and returns the assistant message.
6. On success the assistant bubble is appended and the session list is refreshed. On failure the optimistic user bubble is removed and an error banner is shown above the composer.
7. The send button and input are disabled while generating to prevent duplicate submissions.

API Routes Added

Two new read routes were added alongside the existing [POST /api/chat/message](#):

- `GET /api/chat/sessions` — returns up to 50 sessions for the authenticated user, ordered by `updated_at` descending.
- `GET /api/chat/sessions/{sessionId}` — returns all messages for a single session in chronological order. Returns `404` if the session does not exist or belongs to a different user.

Both routes validate the session with `supabase.auth.getUser()` and return `401` if the user is not authenticated.

How to Test

Start the dev server and log in as an instructor. From the instructor dashboard at `/dashboard/instructor`, click the **AI Chat Assistant** link in the left sidebar. This opens the chat page. Click **+ New Chat**, type a message, and press Enter or click Send. The typing indicator should appear, then the AI response should populate in the thread. The session will then appear in the sidebar. Clicking it again later will reload the full history from the database.

Verification

After sending a message, two tables in Supabase should reflect it: the `chat_sessions` table should contain a new row with the generated session ID and a title derived from the first message, and the `chat_messages` table should contain one row with `role = 'user'` and one with `role = 'assistant'`, both linked to that session ID. Refreshing the page and selecting the session from the sidebar should restore the full conversation from the database.

Session-Level Aggregates for Lecture Analytics

By Jesus Barrios

What Was Built

A `_compute_aggregates` function in `CvResearch/main.py` that takes the list of per-second samples produced during video analysis and returns a summary dictionary. The flat script was also refactored into a callable `analyze_zoom_segment(video_path)` function that runs the full detection loop and stores the aggregates under `result["aggregates"]` before returning.

How It Works

`_compute_aggregates` receives the `samples` list built during the frame loop and computes the following:

1. Count total samples, face-visible samples, and pose-visible samples by filtering on `has_face` and `has_pose`.
2. Compute `face_visibility_ratio` and `pose_visibility_ratio` as the fraction of total samples where each was detected.
3. Sum each blendshape key across only the face-visible samples using `s.get("blendshapes", {})`. Divide each sum by the face sample count to get per-key means. If no face was detected the blendshape dicts are empty and callers should check `face_visibility_ratio == 0` before interpreting them.
4. Sort the blendshape means descending and take the top `top_k` entries (default 5) as `top_blendshapes`.
5. Return the full result dict including `sample_count` and `face_sample_count` as raw counts.

`analyze_zoom_segment` wraps the existing frame loop, calls `_compute_aggregates(samples)` once after the loop, and returns a single dict with `samples`, `aggregates`, `annotated_images`, `backend`, `setup_error`, `fps`, and `total_time_s`.

Output Shape

The `aggregates` dict returned by `_compute_aggregates` has the following structure:

None

```
{
  "face_visibility_ratio": 0.93,
  "pose_visibility_ratio": 0.88,
  "blendshape_means": { "mouthSmile_L": 0.14, ... },
  "top_blendshapes": [
    {"name": "mouthSmile_L", "mean": 0.14},
    {"name": "mouthSmile_R", "mean": 0.13}
  ],
  "sample_count": 300,
  "face_sample_count": 279
}
```

If the OpenCV fallback is active, **blendshapes** is absent from every sample and **blendshape_means** and **top_blendshapes** will both be empty. The visibility ratios still compute correctly in that case.

How to Test

Place a video file at **CvResearch/example_video3.mov** and run the script directly:

None

```
cd CvResearch
```

```
python main.py
```

The script calls **analyze_zoom_segment** and prints **face_visibility_ratio**, **pose_visibility_ratio**, and **top_blendshapes** to stdout alongside the existing backend and timing lines. To inspect the full aggregate dict from another script, import and call the function directly:

None

```
from main import analyze_zoom_segment

result = analyze_zoom_segment("example_video3.mov")

print(result["aggregates"])
```

Verification

After running, the printed output should show `face_visibility_ratio` and `pose_visibility_ratio` as decimals between 0 and 1, and `top_blendshapes` as a list of up to five name/mean pairs. If the tasks backend loaded successfully, blendshape means will be non-empty. If the OpenCV fallback was used, blendshape fields will be empty lists and dicts but the visibility ratios will still be present and correct.

To verify the math manually, count the number of samples in the printed output where `has_face` would be true and divide by `sample_count`. That ratio should match `face_visibility_ratio` exactly. Similarly, `face_sample_count` should equal `sample_count` multiplied by `face_visibility_ratio` with no rounding error since both values are derived from the same integer count.

A quick sanity check for the blendshape means is to confirm that every value in `blendshape_means` falls between 0 and 1, since ARKit blendshape scores are normalized to that range. Any value outside that range would indicate a corrupted sample or a model returning unexpected output. The `top_blendshapes` list should be sorted strictly descending by `mean` and contain at most five entries, or fewer if the face was only detected in a small number of frames with limited blendshape diversity.

Limitations and Open Questions

Pose metrics are currently limited to a presence ratio. Finer upper-body signals such as shoulder angle, head tilt, and gesture frequency require processing the raw landmark coordinates and are out of scope for this task. The `has_pose` flag only indicates whether a person was detected in the frame, not how actively they were moving or whether their posture was engaged.

Blendshape data depends entirely on the FaceLandmarker model from Task 1. The current `FaceDetector` used in this script returns bounding boxes only, so `blendshape_means` and `top_blendshapes` will be empty until FaceLandmarker is

wired in. The aggregate function handles this gracefully and no changes to `_compute_aggregates` will be needed when that swap is made.

Per-second blendshape time series are retained in `samples` alongside the aggregates. This allows callers to reconstruct the full timeline if needed, for example to detect a spike in a particular expression mid-session. Callers that only need the summary can discard `samples` after reading `aggregates` to reduce memory and payload size.

Next Steps

The aggregates produced here are the direct input to the feedback LLM. The next task will pass `face_visibility_ratio`, `pose_visibility_ratio`, and `top_blendshapes` as structured context into the prompt so the model can comment on instructor presence, energy, and delivery in concrete terms rather than generic coaching advice.

Beyond that, adding a per-minute breakdown alongside the session-level summary would allow the LLM to identify specific moments where presence dropped or expression shifted, making feedback more actionable. The `top_k` parameter on `_compute_aggregates` can also be raised by callers that want a wider expression profile fed into the model.

Transcription and Feedback Pipeline Integration

By Jesus Barrios

What Was Built

Extended the `POST /api/feedback/generate` route to accept a `source_type` of `video`. When a video source is specified, the route downloads the instructor's video from Supabase Storage, transcribes it using the OpenAI audio transcription API, and feeds the resulting transcript into the existing RAG feedback pipeline. The output is the same feedback PDF that the PDF path already produces, just sourced from spoken content instead of a written document.

How It Works

The caller adds `source_type: "video"` and `videoFileId` to the existing request body. Here is what the route does:

1. Validate that `instructorId` and `videoFileId` are both present and valid UUIDs.
2. Authenticate the user. Same rule as before: admin or the instructor acting on their own record.
3. List the `videos` bucket at `{instructorId}/` and search for a file whose name starts with `videoFileId` to resolve the extension (`.mp4`, `.mov`, or `.webm`).
4. Download the video into a buffer and hand it to `transcribeVideoBuffer`.
5. `transcribeVideoBuffer` writes the buffer to a temp file under `os.tmpdir()`, calls `client.audio.transcriptions.create` with model `gpt-4o-mini-transcribe`, and deletes the temp file in a `finally` block regardless of outcome.
6. Pass the transcript string to `getFeedbackFromRag` with `source: "video_transcript"`.
7. Render and store the feedback PDF. The `videoFileId` is used as the `lesson_plan_id` grouping key in the feedback row.
8. Return `{ success, feedbackId, storagePath }` exactly like the PDF path does.

Files Changed

New: `apps/web/lib/feedback/transcribe-video.ts` contains `transcribeVideoBuffer`, the TypeScript in-process transcription helper that mirrors the behavior of `backend/transcription.py`.

Modified: `apps/web/lib/feedback/get-feedback-from-rag.ts` now exports a `FeedbackSource` type and accepts an optional `{ source }` second argument. A separate `VIDEO_FEEDBACK_SYSTEM_PROMPT` is used when the source is `video_transcript`, targeting spoken delivery and session-level alignment rather than written lesson plan structure. All existing call sites are unaffected since the argument defaults to `written_lesson_plan`.

Modified: `apps/web/app/api/feedback/generate/route.ts` adds `source_type` and `videoFileId` to the request type, branches validation so video mode does not require `lessonPlanId`, and inserts the video handling block between the auth check and the existing `file_id` block.

How to Test

Upload a video first via `POST /api/videos/upload` to get a `file_id`, then call the feedback route from the browser console while logged in:

None

```
const res = await fetch('/api/feedback/generate', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    source_type: 'video',
    instructorId: '<your-user-uuid>',
    videoFileId: '<file_id-from-upload>',
  }),
})

console.log(res.status, await res.json())
```

A successful response returns `200` with a `feedbackId` and `storagePath`. Missing or invalid UUIDs return `400`. Accessing another user's video returns `403`. A video not found in storage returns `404`.

Backward Compatibility

All existing request shapes continue to work without change. Omitting `source_type` or setting it to anything other than `video` routes through the original PDF logic. The `getFeedbackFromRag` signature change is additive and does not affect any existing callers.